



**Yenepoya Institute of Arts, Science,
Commerce, and Management**

Project – High Level Design

on

**Design and Development of Natural Language
Interface for E-commerce Infrastructure**

Student Name: Alena Latheesh

Industry Mentor: Mr. Shashank

Guided by: Mr. Praveen

Table of Contents

1. Introduction.....	3
1.1. Scope of the document	3
1.2. Intended Audience.....	3
1.3. System overview	4
2. System Design	5
2.1. Application Design.....	6
2.2. Process Flow.....	7
2.3. Information Flow	7
2.4. Components Design.....	9
2.5. Key Design Considerations.....	9
2.6. API Catalogue	10
3. Data Design.....	11
3.1. Data Model.....	11
3.2. Data Access Mechanism.....	11
3.3. Data Retention Policies.....	12
3.4. Data Migration	12
4. Interfaces.....	13
5. State and Session Management	14
6. Caching	15
7. Non-Functional Requirements	16
7.1. Security Aspects.....	16
7.2. Performance Aspects.....	16
8. References.....	17

1. Introduction

This document outlines the High-Level Design (HLD) for the "Design and Development of Natural Language Interface for E-Commerce Infrastructure" project. It defines the system architecture, primary modules, and data flow required to integrate a natural language chatbot and web-based dashboard with a backend e-commerce REST API.

1.1. Scope of the Document

This document defines the architectural boundaries of the system. The scope includes:

- **Presentation & Interaction Layer:** The CLI environment and natural language parsing system.
- **Application Layer:** The centralized REST API acting as the business logic gateway.
- **Persistence Layer:** The file-based JSON data management system.
- **Out of Scope:** Multi-tenant databases and user authentication architectures are explicitly excluded.

1.2. Intended Audience

- ☐ System Architects and Developers
- ☐ Project Evaluators
- ☐ Academic Reviewers

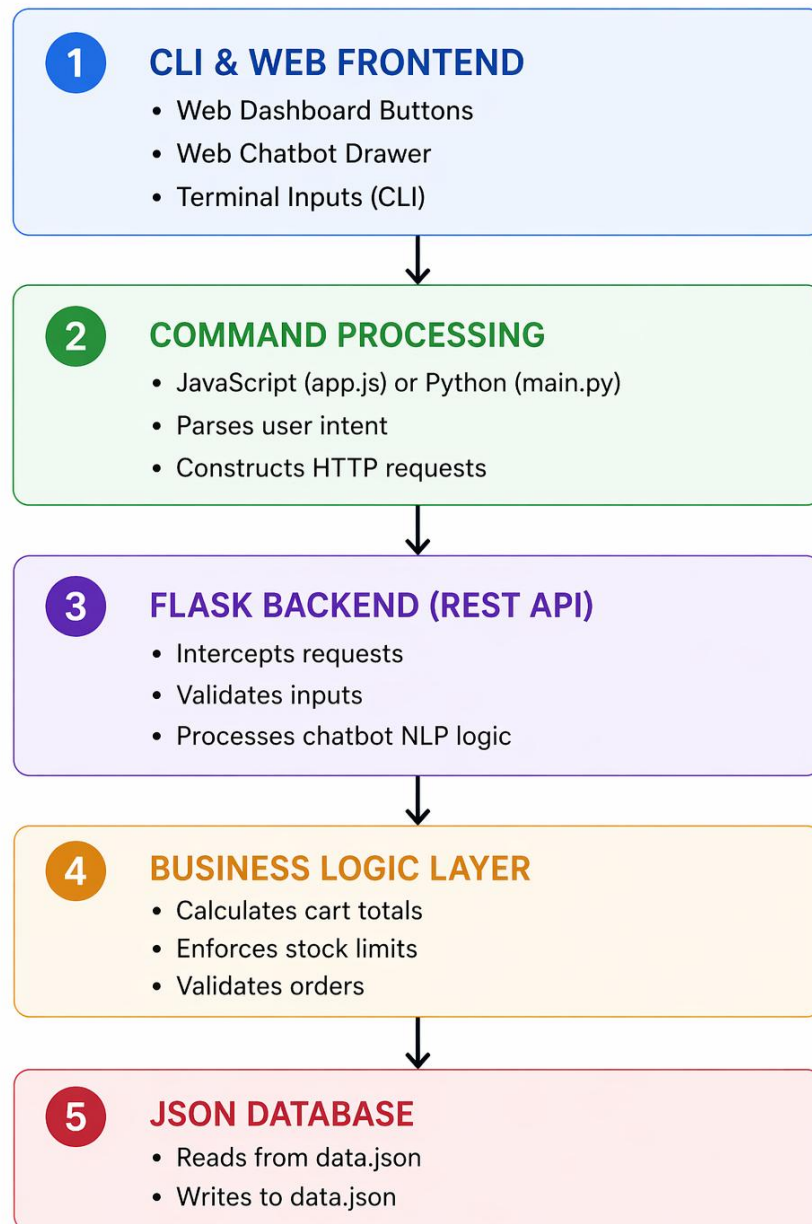
1.3. System Overview

The system is designed as a decoupled, client-server architecture. It provides a hybrid interface, allowing users to browse products, manage a shopping cart, and execute checkouts via a graphical web dashboard or an optional command-line interface. The frontend clients capture user intent and present data, while the backend server acts as the authoritative source for all business rules and state changes.

2. System Design

- Five-layer Architecture:

SYSTEM ARCHITECTURE



- **CLI & WEB FRONTEND:** User interacts via web dashboard buttons, the web chatbot drawer, or terminal inputs.
- **COMMAND PROCESSING:** Frontend scripts (JavaScript app.js or Python main.py) parse user intent and construct HTTP requests.
- **FLASK BACKEND (REST API):** Intercepts requests, validates inputs, and processes chatbot NLP logic.
- **BUSINESS LOGIC LAYER:** Calculates cart totals, enforces stock limits, and validates orders.
- **JSON DATABASE: Reads/Writes to data.json**

2.1 Application Design

- **Client Interface Module (Frontend):** Acts as the presentation layer. It utilizes standard web technologies (HTML/JS) for the primary dashboard and Python for the CLI. It handles client-side validation and HTTP payload construction.
- **API Gateway & Business Logic Module (Backend):** A lightweight web server (app.py) that enforces business rules and processes natural language chatbot queries before orchestrating data updates.
- **Storage Management Module (Data):** An abstracted file I/O system handling the serialization of application state to ensure catalog and cart persistence.

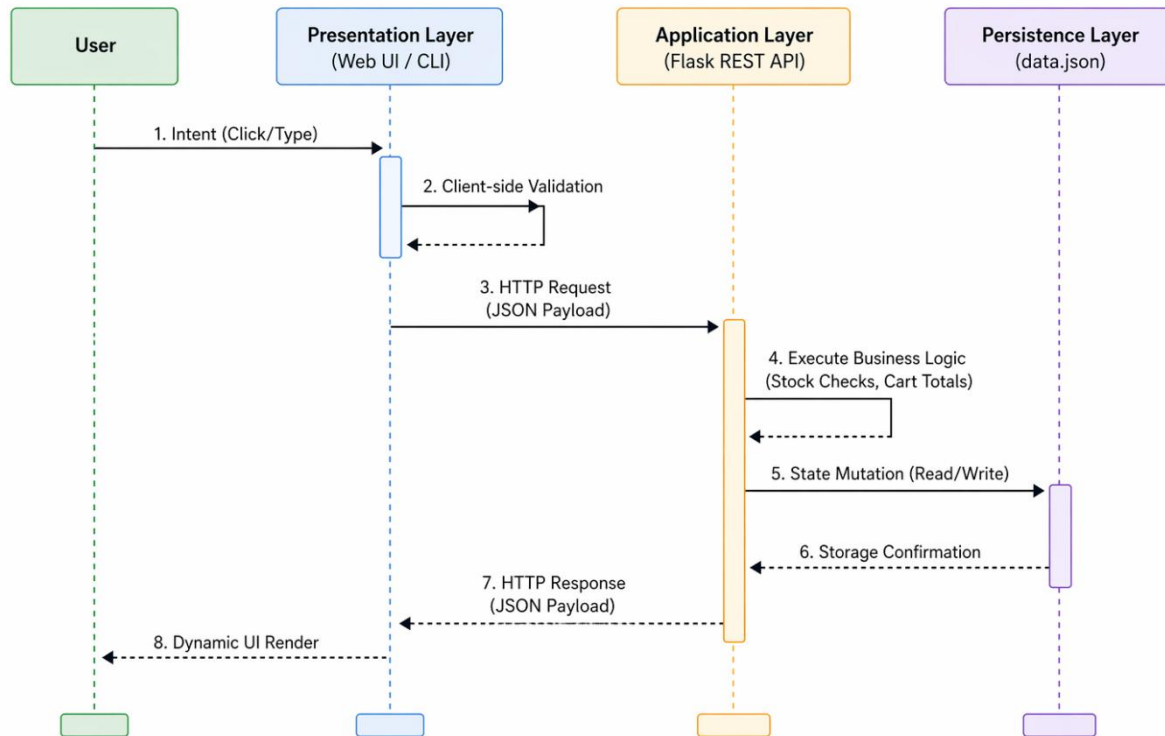
2.2 Process Flow

The system processes user requests through a standardized lifecycle:

- **Intent Capture:** The user interacts with the Web UI (button clicks/chatbot text) or CLI.
- **Payload Construction:** The frontend packages the interaction into a JSON payload.
- **Route Resolution:** The backend receives the HTTP request and routes it to the specific business handler (e.g., /add, /chatbot).
- **State Evaluation:** The backend verifies request constraints against the current data state.
- **State Mutation & Response:** The backend updates the persistent JSON storage and returns a formatted success response for the client to render.

2.3 Information Flow

- **Upstream:** Browser (Fetch API) / CLI (Requests) -> HTTP Request (JSON) -> REST API
- **Processing:** REST API -> Memory -> File Storage
- **Downstream:** REST API -> HTTP Response (JSON) -> Browser DOM / CLI Console

TRANSACTION FLOW SEQUENCE DIAGRAM

2.4 Components Design

- **Presentation Component:** Focuses on user experience, input sanitization, and dynamically rendering JSON data into HTML DOM elements or console text.
- **Controller Component:** The API router mapping HTTP endpoints to Python business functions.
- **Data Access Component:** Isolates file reading/writing operations from core logic to prevent corruption.

2.5 Key Design Considerations

- **Stateless Communication:** The server retains no client state in memory; every HTTP request provides full context.
- **Data Integrity:** The storage file is immediately updated after any successful transaction.
- **Decoupling:** The REST API is completely agnostic to whether the client is a web browser or a terminal script.

2.6 API Catalogue

- ☐ / (GET): Serves the Web UI index.html.
- ☐ /dashboard-data (GET): Fetches complete overview, cart, and product data for web rendering.
- ☐ /products, /cart (GET): Retrieves catalog or cart state.
- ☐ /add (POST): Validates and processes cart additions.
- ☐ /checkout (GET): Processes order completion.
- ☐ /chatbot (POST): Evaluates natural language intent and returns structured dialog responses.

3 Data Design

The system employs a flat-file NoSQL approach for data persistence, acting as a lightweight, persistent database. This choice intentionally bypasses the overhead of traditional relational database management systems (RDBMS) to prioritize speed and architectural simplicity

3.1 Data Model

The system utilizes a document-based data structure consisting of distinct collections. Rather than using rigid tables with strict schemas, the data is stored as hierarchical, easily readable JSON objects.

- **Products Collection:** This acts as the inventory catalog. It stores both *static attributes* (like product identifications, names, delivery times, and pricing) and *dynamic attributes* (like available stock levels, which change as orders are processed).
- **Cart Collection:** This acts as the transactional state. It stores active user selections by mapping the chosen product identifiers to their requested quantities

3.2 Data Access Mechanism

Because the system uses a stateless REST API, data is managed via a centralized file-handling mechanism that prioritizes data freshness.

- **In-Memory Processing:** During an active request (such as a user asking to view the cart or checkout), the system loads the entire dataset from the disk directly into the server's active memory. This allows for rapid querying, price calculations, and stock validation without repetitive disk reads.
- **Immediate Serialization:** To guarantee data integrity and prevent "lost" cart items, the memory state is strictly temporary. The system serializes the updated state and writes it back to the disk immediately following any state-altering transaction (such as a successful cart addition or checkout).

3.3 Data Retention Policies

Data within the system follows two distinct lifecycles based on its operational role:

- **Inventory Data (Permanent):** The product catalog and its associated data are retained permanently. The stock values are only altered downward during authorized, validated stock deductions.
- **Transactional Data (Transient):** The cart data is retained persistently between application sessions, meaning a user will not lose their cart if the server reboots. However, its ultimate lifecycle is temporary; the system is programmed to automatically purge and reset the cart array upon the successful execution of the checkout workflow.

3.4 Data Migration

Data migration architectures are not required for this iteration of the system. Because this is a greenfield, single-user demonstration environment relying on a self-contained JSON file, there are no legacy databases or historical user records that need to be mapped, transformed, or ported into the current architecture

4 Interfaces

The system is designed with strict boundaries between its distinct architectural layers, communicating entirely through standardized, decoupled interfaces.

- **Web-to-Server Interface:** The primary boundary between the graphical user environment and the backend logic is bridged via a RESTful API. Communication is handled over the HTTP protocol utilizing the JavaScript Fetch API, with JSON serving as the universal data interchange format. This design guarantees that the frontend remains completely lightweight, acting only as a presentation layer that is entirely agnostic to how the backend processes or stores the data.
- **CLI-to-Server Interface:** Operating on the exact same principles as the Web UI, the terminal-based client communicates with the centralized REST API over HTTP using the Python requests library. This dual-interface capability proves the high level of decoupling in the architecture; the server does not differentiate between a web browser and a terminal client.
- **Server-to-Storage Interface:** The Flask API communicates with the persistent storage layer via local file system I/O streams. The system abstracts the physical reading and writing processes (utilizing Python's native json library), ensuring that the core business logic components only interact with standardized, deserialized data objects in memory rather than managing raw file streams directly.

5 State and Session Management

- **State Management:** The architectural paradigm relies on a strictly stateless API backend. The server does not maintain any transient memory of previous interactions between HTTP requests. All application state—most critically the current contents of the user's shopping cart—is delegated entirely to the persistent data layer (data.json). This approach ensures high fault tolerance; even if the server process is unexpectedly restarted, no transactional state is lost.
- **Session Management:** The system is explicitly designed as a single-user demonstration environment to isolate and demonstrate core e-commerce transaction logic. Consequently, concurrent session management, user authentication protocols, and authorization scopes have been bypassed by design to reduce unnecessary architectural complexity.

6 Caching

Caching mechanisms (such as Redis, Memcached, or in-memory application caches) are explicitly not implemented in the current system architecture. Every time an API endpoint is triggered, the backend performs a direct disk I/O operation to read the database.

Design Rationale: For a localized simulation with a lightweight dataset, the performance overhead of reading a JSON file directly from the local disk is negligible. Introducing a caching layer would add unnecessary complexity, such as cache invalidation logic.

Strict Data Freshness: By reading directly from the persistent database on every single request, the system completely avoids "stale data" synchronization issues. This architectural choice guarantees that the business logic layer always validates transactions against the absolute latest inventory stock before approving an order.

7 Non-Functional Requirements

7.1 Security Aspects

While strict user authentication is out of scope, the system implements robust architectural security through rigorous data validation at the API gateway:

- **Payload Verification:** The backend gateway enforces strict schema checks, automatically rejecting malformed JSON payloads or requests that are missing required attributes.
- **Business Logic Guards:** The system acts as a strict gatekeeper, preventing illogical state changes (e.g., rejecting requests for zero or negative item quantities, or blocking cart additions that exceed the current available inventory limits).

7.2 Performance Aspects

- **Efficiency:** The decoupled nature of the client interfaces (Web/CLI) combined with a lightweight server framework yields extremely low-latency responses for the end-user.
- **Scalability:** The current architecture utilizes synchronous file I/O operations. While highly efficient for a localized, single-user demonstration, this architectural choice inherently limits horizontal scalability for high-concurrency production environments.
- **Resilience:** The system bounds all network communications with timeout protocols and graceful error handling. This ensures that server unavailability or network drops do not result in a catastrophic crash of the client interfaces.

8 References

- Conceptual System Design Architectures
- Client-Server Interaction Models
- RESTful API Design Principles
- JSON Data Serialization Standards